

PRAKTIKUM SISTEM OPERASI

MODUL 11



Oleh:

Resita Istantia Purwanto

2311104037

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

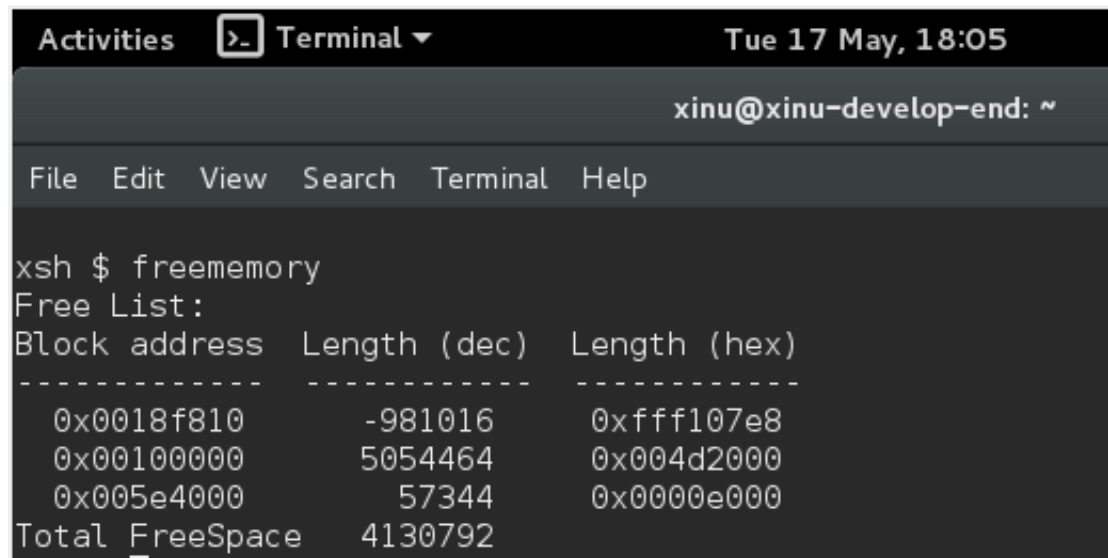
UNIVERSITAS TELKOM

2026

JURNAL/TP

1. [80 poin] Buatlah perintah baru bernama `freememory` yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang diinginkan:



```
Activities Terminal Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018f810     -981016    0xffff107e8
0x00100000      5054464    0x004d2000
0x005e4000       57344     0x0000e000
Total FreeSpace 4130792
```

Pada tugas ini dibuat sebuah perintah shell baru bernama `freememory` yang berfungsi untuk menampilkan informasi mengenai memori bebas (*free memory*) yang terdapat pada sistem Xinu. Perintah ini memiliki dua fungsi utama, yaitu:

- Menampilkan seluruh blok memori bebas yang tersimpan pada *free memory list*. Informasi yang ditampilkan meliputi alamat awal (*block address*), ukuran blok dalam bentuk desimal (*Length (dec)*), dan ukuran blok dalam bentuk heksadesimal (*Length (hex)*).
- Menghitung total kapasitas memori bebas dengan menjumlahkan ukuran seluruh blok yang terdapat pada *free memory list*. Hasil penjumlahan tersebut kemudian ditampilkan sebagai *Total FreeSpace*.

Implementasi perintah dilakukan dengan melakukan penelusuran (*traversal*) pada linked list yang menyimpan daftar blok memori bebas. Setiap node pada list dibaca satu per satu untuk memperoleh alamat dan ukuran blok, kemudian ukurannya diakumulasikan hingga seluruh list selesai diproses. Saat perintah `freememory` dijalankan pada terminal Xinu, sistem akan menampilkan daftar blok memori yang masih tersedia beserta ukuran masing-masing blok. Setelah seluruh blok ditampilkan, program menghitung dan menampilkan total ruang memori bebas yang dimiliki sistem.

Free List:		
Block address	Length (dec)	Length (hex)

0x0018f810	...	
0x00100000	...	
0x0050e400	...	
Total FreeSpace 4130792		

2. Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

:

- Pemisahan ini dilakukan demi efisiensi ukuran file *executable* (biner) saat disimpan di disk atau sebelum dimuat ke memori:
- Data Segment menyimpan variabel global dan statis yang sudah diinisialisasi dengan nilai awal tertentu (misal: `int x = 10;`). Nilai ini harus disimpan di dalam file biner.
- BSS Segment menyimpan variabel global dan statis yang belum diinisialisasi atau diinisialisasi ke nol (misal: `int y[1000];`). File biner tidak perlu menyimpan ribuan angka nol; ia cukup mencatat *ukuran total* yang dibutuhkan BSS. Saat sistem *booting*, Xinu tinggal memesan ruang sebesar itu di memori dan mengosongkannya (di-set ke 0).

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Jawaban :

- Alokasi (*getmem* / *getstk*): Mengambil ruang dari *memlist*. Proses ini mengurangi ukuran total free space dan dapat memecah satu blok memori besar menjadi blok-blok kecil (*fragmentasi*).
- Dealokasi (*freemem* / *freestk*): Mengembalikan memori yang sudah selesai digunakan ke *memlist*. Proses ini menambah ukuran total free space. Sistem Xinu juga mencoba menggabungkan (*coalesce*) blok yang dikembalikan dengan blok kosong di sebelahnya jika alamatnya berurutan, guna mencegah *fragmentasi* yang terlalu parah.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Jawaban:

- Manajemen Manual & Risiko Kebocoran: Memori di *stack* dialokasikan dan didealokasikan secara otomatis saat fungsi dipanggil dan selesai (*LIFO structure*). Sebaliknya, *heap* dikelola secara manual oleh programmer. Jika lupa membebaskan memori (*memory leak*), sistem akan kehabisan memori.
- Fragmentasi Memori: Alokasi dan dealokasi *heap* dengan ukuran yang acak dan waktu yang tidak menentu menyebabkan memori terpecah-pecah menjadi potongan kecil yang berserakan (*fragmentasi eksternal*). Akibatnya, alokasi berikutnya bisa gagal meskipun total free space sebenarnya masih mencukupi.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Jawaban:

- Kebutuhan Ukuran Dinamis: Jumlah dan ukuran blok memori bebas selalu berubah-ubah selama sistem berjalan. *Linked list* memungkinkan penyisipan (saat memori dibebaskan) dan penghapusan (saat memori diambil) secara dinamis tanpa perlu menggeser elemen memori lain.
- Efisiensi Ruang (Zero Overhead Space): Struktur data *linked list* ini disimpan langsung di dalam blok memori bebas itu sendiri (menggunakan *pointer mnext* di dalam ruang kosong tersebut). Jadi, Xinu tidak membutuhkan memori tambahan eksternal untuk melacak di mana saja area yang kosong.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban:

- Tidak Ada Garbage Collection atau Virtual Memory: Xinu adalah sistem operasi *embedded/real-time* yang sederhana. Tidak ada mekanisme otomatis untuk membersihkan memori yang telantar, dan tidak ada *paging* (memori virtual) untuk memindahkan blok memori guna menyatukan ruang kosong.
- Fragmentasi Eksternal yang Tinggi: Karena keterbatasan di atas, jika aplikasi sering melakukan alokasi dan dealokasi dalam ukuran kecil secara acak, *heap* akan cepat mengalami fragmentasi berat. Hal ini dapat menyebabkan kegagalan sistem (*crash* atau *out of memory*) pada sistem yang harus menyala terus-menerus.